

6

Advanced Natural Language Processing

In the previous chapter, we covered the basics of **natural language processing** (NLP). We covered simple representations of text in the form of the bag-of-words model, and more advanced word embedding representations that capture the semantic properties of the text. This chapter aims to build upon word representation techniques by taking a more model-centric approach to text processing. We will go over some of the core models, such as **recurrent neural networks** (RNNs) and **Long Short-Term Memory** (LSTM) networks. We will specifically answer the following questions:

- What are some core deep learning models for understanding text?
- What core concepts form the basis for understanding RNNs?
- What core concepts form the basis for understanding LSTMs?
- How do you implement basic functionality of an LSTM using TensorFlow?
- What are some of the most popular text processing applications of an RNN/LSTM?

Deep learning for text

We have seen various different techniques so far, which employ variants of neural networks for text processing. Word-based embedding is one such common application of neural networks. As seen in the previous chapter, word-based embedding techniques are feature-level, or representation learning, problems. In other words, they solve a very specific problem: Given a text block, represent it in some feature form that is used for a downstream text mining application, such as classification, machine translation, attribute labeling, and so on. A number of machine learning techniques exist today that can apply text mining at varying accuracy levels. In this chapter, we focus on an entirely different model of text processing. We look into the core deep learning models that are suited to text processing applications and can perform both:

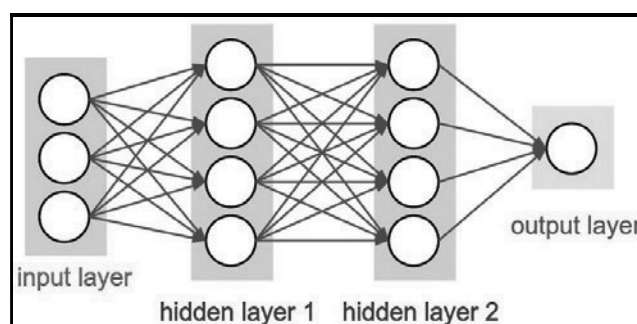
- Representation learning or feature extraction
- Downstream text mining application in a unified model

Before we begin our understanding of deep learning models for text, let's revisit neural networks for a moment and try to understand why they are not suited for some of the important text mining applications.

Limitations of neural networks

Neural networks are a very potent tool for approximating any non-linear function—a problem that arises very frequently in any pattern recognition or machine learning task. Though they are very powerful in their modeling approach, they have certain limitations, which makes them limited in their abilities for various pattern recognition tasks. A few of those limitations are:

- **Fixed sized inputs:** A neural network architecture has a fixed number of input layers. As such, it can only take a fixed sized input and output for any task. This is a limiting factor for many pattern recognition tasks. For example, imagine an image captioning task, where the goal of the network is taking an image and generating words as captions. A typical neural network cannot model this task as for every image, a number of words in the captions are going to be different. Given a fixed output size, it is not possible for a neural network to efficiently model this task. Another example is the task of sentiment classification. In this task, a network should take a sentence as its input and output a single label (for example, positive or negative). Given a sentence has a varying number of words, the input for this task has a variable number of inputs. Hence, a typical neural network cannot model this task. This type of task is often referred to as a *sequence classification* task.
- **Lack of memory:** Another limitation of neural networks is their lack of memory. For example, for the task of sentiment classification, it is important to remember the sequence of words to classify the sentiment of the whole sentence. For a neural network, each input unit is processed independently of each other. As such, the next word token in the sentence has no correlation to any previous word token in the sentence, which makes the task of classifying the sentence extremely difficult. A good model that can perform well at such tasks needs to maintain context or memory:

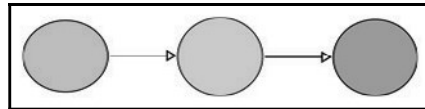


Fixed sized inputs of neural networks (Source: https://raw.githubusercontent.com/cs231n/cs231n.github.io/master/assets/nn1/neural_net2.jpeg)

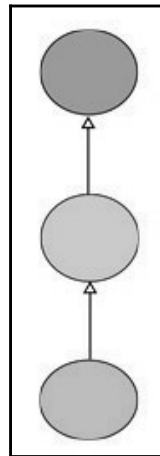
To address these limitations, an RNN is used. This class of deep learning model is our primary focus in this chapter.

Recurrent neural networks

The basic idea behind recurrent neural networks is the vectorization of data. If you look at figure *Fixed sized inputs of neural networks*, which represents a traditional neural network, each node in the network accepts a scalar value and generates another scalar value. Another way to view this architecture is that each layer in the network accepts a vector as its input and generates another vector as its output. Figure *Neural network horizontally rolled up* and figure *Neural network vertically rolled up* illustrate this representation:



Neural network horizontally rolled up



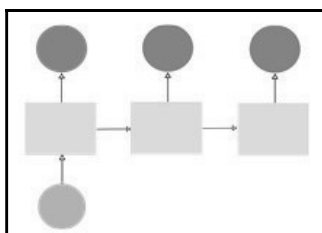
Neural network vertically rolled up

The figure *Neural network vertically rolled up* is a simple RNN representation, which is a one-to-one RNN; one input is mapped to one output using one hidden layer.

RNN architectures

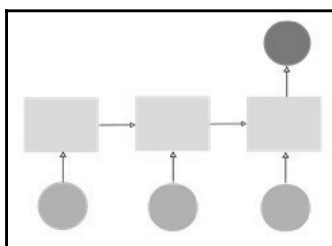
Typically, RNNs have many different architectures. In this section, we will go over some basic architectures of RNNs and discuss how they fit various, different text mining applications:

- **One-to-many RNN:** Figure *RNN: One-to-many architecture* illustrates the basic idea of a one-to-many RNN architecture. As shown in the following figure, in this architecture a single input unit of RNN is mapped to multiple hidden units as well as multiple output units. One application example of this architecture is image captioning. As described previously, in this application the input layer accepts a single image and maps it to multiple words in the caption:



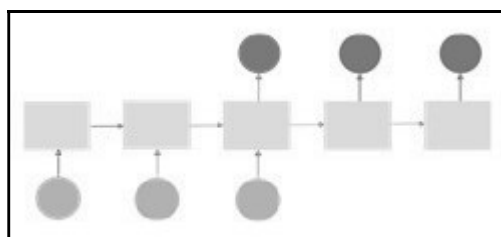
RNN: One-to-many architecture

- **Many-to-one RNN:** Figure *RNN: Many-to-one architecture*, illustrates the basic idea of a many-to-one RNN architecture. As shown in the following figure, in this architecture multiple input units of RNN are mapped to multiple hidden units but only a single output unit. One application example of this architecture is sentiment classification. As described previously, in this application the input layer accepts multiple word tokens from a sentence and maps them to a single sentiment of the sentence as positive or negative:



RNN: Many-to-one architecture

- **Many-to-many RNN:** Figure *RNN: Many-to-many architecture* illustrates the basic idea of a many-to-many RNN architecture. As shown in the following, in this architecture multiple input units of RNN are mapped to multiple hidden units and multiple output units. One application example of this architecture is a machine translation. As described previously, in this application the input layer accepts multiple word tokens from a source language and maps them to multiple word tokens from a target language:



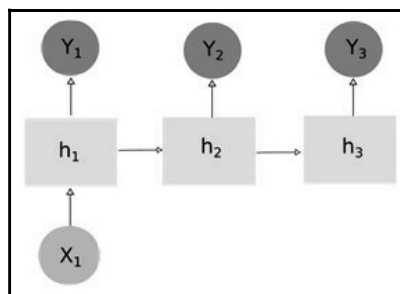
RNN: Many-to-many architecture

Basic RNN model

Figure *Basic RNN model* describes a basic RNN model in more detail. As you can see, this is a simple one-to-many RNN model. If you just focus on nodes X_t , h_t , and Y_t , they are very similar to a one-layer neural network. The only additions to this RNN model are the steps in time when hidden neurons take on different values, such as h_2 and h_3 . The overall sequence of operations in this RNN model is as follows:

- Time step t1 where X_1 is input to the RNN model
- Time step t1 where h_1 is computed based on input X_1
- Time step t1 where Y_1 is computed based on input h_1
- Time step t2 where h_2 is computed based on input h_1
- Time step t2 where Y_2 is computed based on input h_2
- Time step t3 where h_3 is computed based on input h_2
- Time step t3 where Y_3 is computed based on input h_3

Using this model, we can generate multiple output vectors based on each time step. Hence, such models are widely applicable to a number of time-series or sequence-based data models.

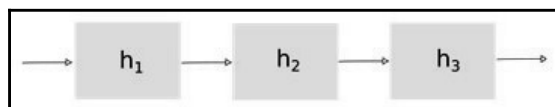


Basic RNN model

Training RNN is tough

In this section, we will look at some of the existing limitations of training RNN. We will also dive deeper and understand why it is tough to train RNN.

The traditional method of training neural networks is through the backpropagation algorithm. In the case of RNN, we need to perform a backpropagation of gradient through time, often referred to as **backpropagation through time (BPTT)**. Although it's numerically possible to compute backward propagation of gradients through time, it often results in poor results due to the classic vanishing (or exploding) gradient problem as shown in figure *vanishing gradient problem in RNN with multiplicative gradients*:



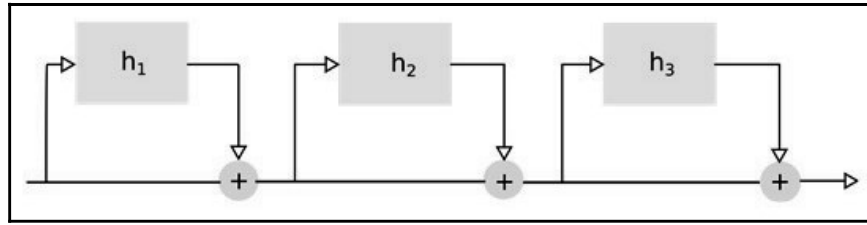
Vanishing gradient problem in RNN with multiplicative gradients

To understand this concept in more detail, let's take a look at figure *Basic RNN model*. It shows a single layer of hidden neurons through three-time steps. To compute a forward pass of the gradient across this, we need to compute a derivative of a composite function as follows:

$$H = h_3(h_2(h_1(x)))$$

$$\frac{\partial H}{\partial x} = \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial x}$$

As you can imagine, each gradient computation here successively becomes smaller, and multiplying a smaller value with a bigger value leads to diminishing the overall gradient when computed through a large time step. Hence RNNs cannot be trained in an efficient manner with longer time steps using this approach. One way to solve this approach is to use a gating logic as shown in figure *Solution to vanishing gradient problem with additive gradients*:



Solution to vanishing gradient problem with additive gradients

In this logic, instead of multiplying the gradients together, we add them as follows:

$$H = h_1(x) + h_2(x) + h_3(x)$$

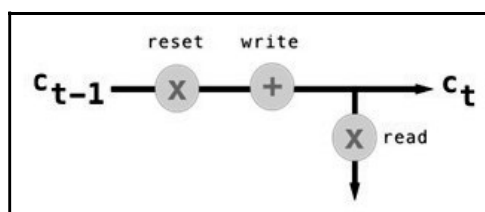
$$\frac{\partial H}{\partial x} = \frac{\partial h_1}{\partial x} + \frac{\partial h_2}{\partial x} + \frac{\partial h_3}{\partial x}$$

As can be seen from the preceding equation, the overall gradient can be computed as a sum of smaller gradients, which do not diminish even when passed through longer time steps. This addition is achieved due to the gating logic, which adds the output of the hidden layer with the original input at every time step, thereby reducing the impact of diminishing gradients. This gating architecture forms the basis of a new type of RNN also known as Long Short-Term Memory networks or LSTMs. LSTMs are the most popular way to train RNNs on a long sequence of temporal data and have been shown to perform reasonably well on a wide variety of text mining tasks.

Long short-term memory network

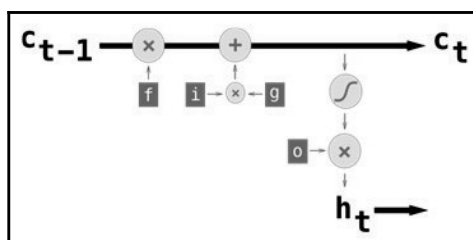
So far, we have seen that RNNs perform poorly due to the vanishing and exploding gradient problem. LSTMs are designed to help us overcome this limitation. The core idea behind LSTM is a gating logic, which provides a memory-based architecture that leads to an additive gradient effect instead of a multiplicative gradient effect as shown in the following figure. To illustrate this concept in more detail, let us look into LSTM's memory architecture. Like any other memory-based system, a typical LSTM cell consists of three major functionalities:

- Write to memory
- Read from memory
- Reset memory



LSTM: Core idea (Source: <https://ayearofai.com/rohan-lenny-3-recurrent-neural-networks-10300100899b>)

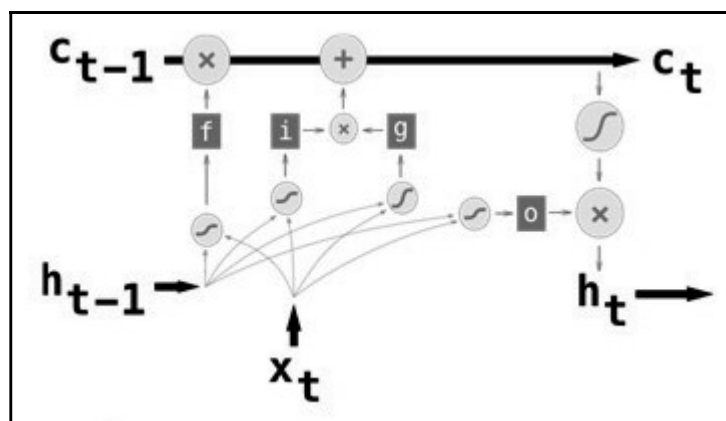
Figure *LSTM: Core idea* illustrates this core idea. As shown in the figure LSTM: Core idea, first the value of a previous LSTM cell is passed through a reset gate, which multiplies the previous state value by a scalar between 0 and 1. If the scalar is close to 1, it leads to the passing of the value of the previous cell state (remembering the previous state). In case it is closer to 0, this leads to blocking of the value's previous cell state (forgetting the previous state). Next, the write gate simply writes the transformed output of the reset gate. Finally, the read gate reads a view of the output from the write gate:



LSTM: Gating functions in the cell (Source: <https://ayearofai.com/rohan-lenny-3-recurrent-neural-networks-10300100899b>)

Understanding LSTM's gating logic can be quite complex. To describe this more succinctly, let us take a detailed look into the cell architecture of LSTM without any inputs as shown in figure *LSTM: Core idea*. The gating functions now have well-defined labels. We can call them as follows:

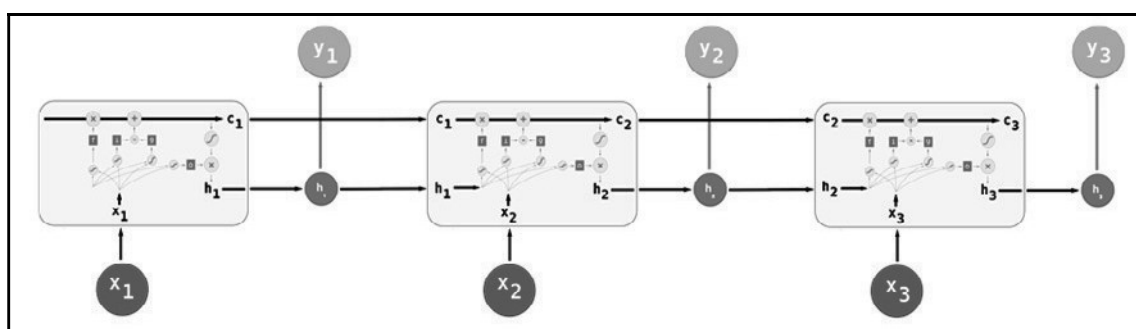
1. **f gate**: Often referred to as the forget gate, this gate applies a sigmoid function to the input cell value from a previous time step. Since a sigmoid function takes any value between 0 and 1, this gate amounts to forgetting a portion of the previous cell state value based on the activation of the sigmoid function.
2. **g gate**: The primary function of this gate is to regulate the additive factor of the previous cell state value. In other words, the output of the f gate is now added with a certain scalar controlled by the g gate. Typically, a tanh function between -1 and 1 is applied in this case. As such, this gate often acts as an incrementing or decrementing a counter.
3. **i gate**: Though g gates regulate the additive factor, the *i* gate is another tanh function between 0 and 1 that dictates what portion of the g gate can actually be added to the output of the f gate.
4. **o gate**: Also known as the *output gate*, this gate also uses a sigmoid function to generate a scaled output, which is then inputted to the hidden state in the current time step:



LSTM: Basic cell architecture (Source: <https://ayearofai.com/rohan-lenny-3-recurrent-neural-networks-10300100899b>)

Figure *LSTM: Gating functions in the cell* shows the full LSTM cell with both inputs and hidden states from previous and current time steps. As shown here, each of the preceding four gates receives inputs from both the hidden state from the previous time step as well as input from the current time step. The output of the cell is passed to the hidden state of the current time step, as well as carried forward to the next LSTM cell. Figure *End-to-end LSTM network* describes this connection visually.

As shown here, each LSTM cell acts as a separate unit between the input neuron and hidden neuron across all time steps. Each of these cells is connected across time steps using a two-channel communication mechanism sharing both the LSTM cell output as well as hidden neuron activations across different time steps:

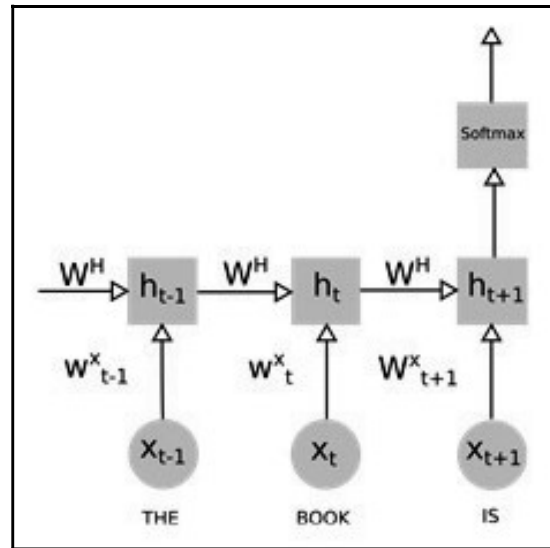


End-to-end LSTM network (Source: <https://ayearofai.com/rohan-jenny-3-recurrent-neural-networks-10300100899b>)

LSTM implementation with tensorflow

In this section, we will look at an example of using LSTM in TensorFlow for the task of sentiment classification. The input to LSTM will be a sentence or sequence of words. The output of LSTM will be a binary value indicating a positive sentiment with 1 and a negative sentiment with 0. We will use a many-to-one LSTM architecture for this problem since it maps multiple inputs onto a single output. Figure *LSTM: Basic cell architecture* shows this architecture in more detail. As shown here, the input takes a sequence of word tokens (in this case, a sequence of three words). Each word token is input at a new time step and is input to the hidden state for the corresponding time step.

For example, the word *Book* is input at time step t and is fed to the hidden state h_t :



Sentiment analysis using LSTM

To implement this model in TensorFlow, we need to first define a few variables as follows:

```
batch_size = 4
lstm_units = 16
num_classes = 2
max_sequence_length = 4
embedding_dimension = 64
num_iterations = 1000
```

As shown previously, `batch_size` dictates how many sequences of tokens we can input in one batch for training. `lstm_units` represents the total number of LSTM cells in the network. `max_sequence_length` represents the maximum possible length of a given sequence. Once defined, we now proceed to initialize TensorFlow-specific data structures for input data as follows:

```
import tensorflow as tf

labels = tf.placeholder(tf.float32, [batch_size, num_classes])
raw_data = tf.placeholder(tf.int32, [batch_size, max_sequence_length])
```

Given we are working with word tokens, we would like to represent them using a good feature representation technique. We propose using word embedding techniques from the Chapter 5, *NLP- Vector Representation*, for this task. Let us assume the word embedding representation takes a word token and projects it onto an embedding space of dimension, `embedding_dimension`. The two-dimensional input data containing raw word tokens is now transformed into a three-dimensional word tensor with the added dimension representing the word embedding. We also use pre-computed word embedding, stored in a `word_vectors` data structure. We initialize the data structures as follows:

```
data = tf.Variable(tf.zeros([batch_size, max_sequence_length,
                             embedding_dimension]), dtype=tf.float32)
data = tf.nn.embedding_lookup(word_vectors, raw_data)
```

Now that the input data is ready, we look at defining the LSTM model. As shown previously, we need to create `lstm_units` of a basic LSTM cell. Since we need to perform a classification at the end, we wrap the LSTM unit with a dropout wrapper. To perform a full temporal pass of the data on the defined network, we unroll the LSTM using a `dynamic_rnn` routine of TensorFlow. We also initialize a random weight matrix and a constant value of 0.1 as the bias vector, as follows:

```
weight = tf.Variable(tf.truncated_normal([lstm_units, num_classes]))
bias = tf.Variable(tf.constant(0.1, shape=[num_classes]))
lstm_cell = tf.contrib.rnn.BasicLSTMCell(lstm_units)
wrapped_lstm_cell = tf.contrib.rnn.DropoutWrapper(cell=lstm_cell,
                                                    output_keep_prob=0.8)
output, state = tf.nn.dynamic_rnn(wrapped_lstm_cell, data,
                                   dtype=tf.float32)
```

Once the output is generated by the dynamic unrolled RNN, we transpose its shape, multiply it by the weight vector, and add a bias vector to it to compute the final prediction value:

```
output = tf.transpose(output, [1, 0, 2])
last = tf.gather(output, int(output.get_shape()[0]) - 1)
prediction = (tf.matmul(last, weight) + bias)
weight = tf.cast(weight, tf.float64)
last = tf.cast(last, tf.float64)
bias = tf.cast(bias, tf.float64)
```

Since the initial prediction needs to be refined, we define an objective function with cross-entropy to minimize the loss as follows:

```
loss = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits
                      (logits=prediction, labels=labels))
optimizer = tf.train.AdamOptimizer().minimize(loss)
```

After this sequence of steps, we have a trained, end-to-end LSTM network for sentiment classification of arbitrary length sentences.

Applications

Today, RNNs (for example, LSTM) have been used in a variety of different applications ranging from time series data modeling, image classification, and video captioning, as well as textual analysis. In this section, we will cover some important applications of RNNs for solving different natural language understanding problems.

Language modeling

Language modeling is one of the fundamental problems in **natural language understanding** (NLU). The core idea of a language model is to model important distributional properties of the words in a given language. Once such a model is learnt, it can be applied to a sequence of new words to generate the most likely next word token given the learned distributional representation. More formally, a language model computes a joint probability over a sequence of words as follows:

$$P(W_1, W_2, W_3, \dots, W_n) = \prod_i P(W_i | W_1, W_2, \dots, W_{i-1})$$

Estimating this probability is computationally expensive, hence a number of estimation techniques exist, which make certain assumptions about the time range dependence of the word tokens. Some of them are as follows:

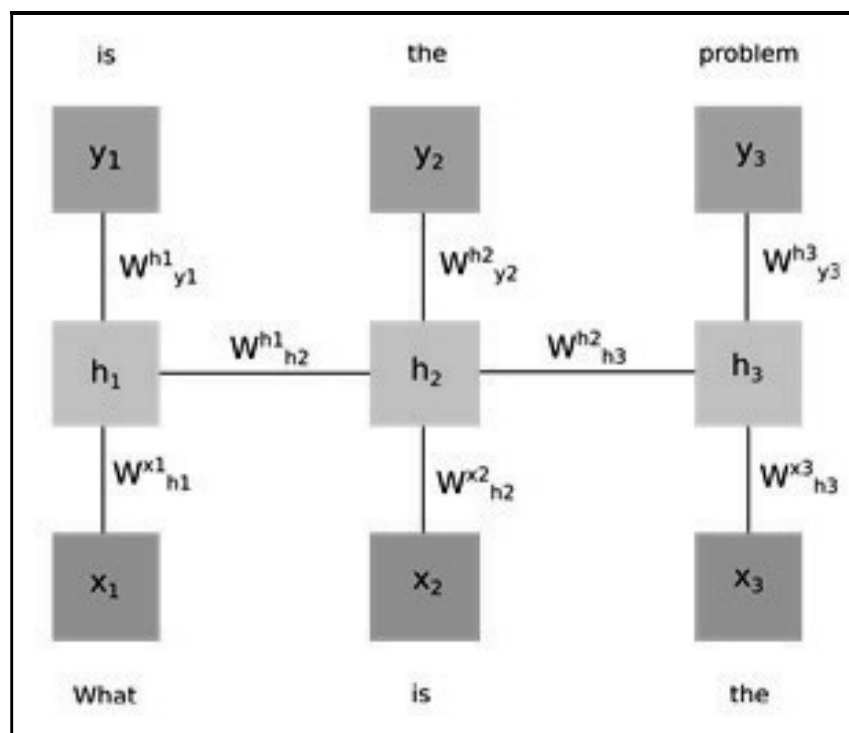
- **Unigram model:** It assumes that each word token is independent of the sequence of words before and after it:

$$P(W_1, W_2, W_3, \dots, W_n) \approx \prod_i P(W_i)$$

- **Bigram model:** It assumes that each word token is dependent on only the word token immediately before it:

$$P(W_1, W_2, W_3, \dots, W_n) = \prod_i P(W_i | W_{i-1})$$

We can solve the language model estimation problem efficiently through the use of an LSTM-based network. The following figure illustrates a specific architecture that estimates a three-gram language model. As shown here, we take a many-to-many LSTM and chunk the whole sentence into a running window of three-word tokens each. For example, let us assume a training sentence is: [**What, is, the, problem**]. The first input sequence is: [**What, is, the**] and the output is [**is, the, problem**]:

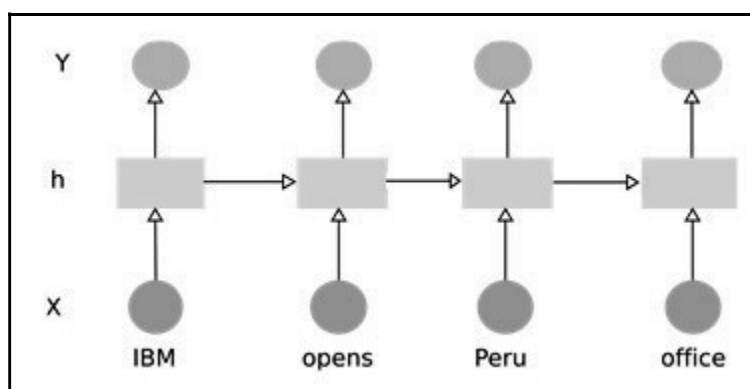


Language modeling with traditional LSTM

Sequence tagging

Sequence tagging can be understood as a problem where the model sees a sequence of words or tokens for each word in the sequence and it is expected to emit a label. In other words, the model is expected to tag the whole sequence of tokens with an appropriate label from a known label dictionary. Sequence tagging has some very interesting applications in natural language understanding such as named entity recognition, part of speech tagging, and so on:

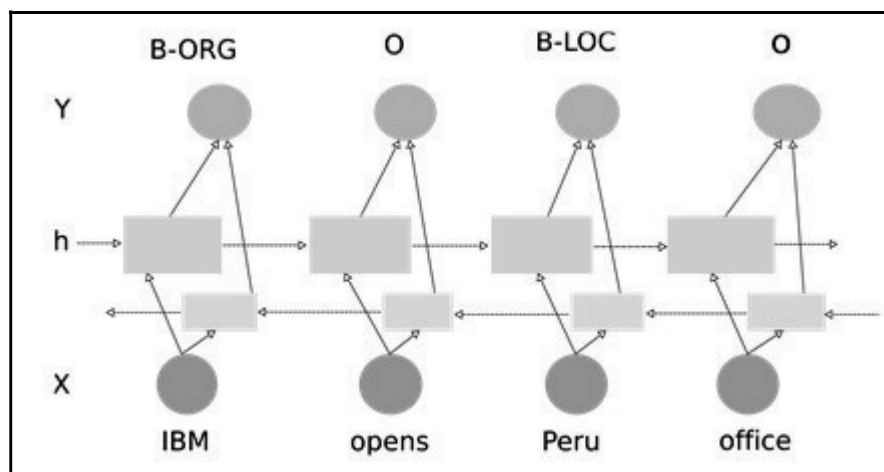
- **Named Entity Recognition:** Also known as **NER**, Named Entity Recognition is an information extraction technique that aims to recognize named entities in the given sequence of text tokens (for example, words). Some of the common named entities include person, location, organization, currency, and so on. For example, when you input a sequence *IBM opens an office in Peru* to an NER system, it can recognize the presence of two named entities *B-ORG* and *B-LOC*, and label them appropriately to tokens *IBM* and *Peru* respectively.
- **Part of Speech tagging:** Also known as POS tagging, POS tagging is an information extraction technique that aims to recognize parts of speech in text. Some of the common parts of speech classes are NN (noun, singular), NNS (noun, plural), NNPS (proper noun, singular), VB (verb, base form), CC (coordinating conjunction), CD (cardinal number), and so on:



Sequence tagging with traditional LSTM

As shown in the preceding figure, *Sequence Tagging with Traditional LSTM*, one can model this problem with a simple LSTM. One important point to note from this figure is LSTM is only able to make use of the previous context of data. For example, in hidden state at time t , the LSTM cell sees input from time t and output from hidden state at time $t-1$. There is no way to make use of any future context from time greater than t in this architecture. This is a strong limitation of traditional LSTM models for the task of sequence tagging.

To address this issue, bi-directional LSTM or B-LSTMs are proposed. The core idea of a bi-directional LSTM is to have two LSTM layers, one in a forward direction and another in a backward direction. With this design change, you can now combine information from both directions to get previous context (forward LSTM) and future context (backward LSTM). Figure *Language modeling with traditional LSTM* shows this design in more detail. B-LSTMs are one of the most popular LSTM variants used for sequence tagging tasks today:



Sequence tagging with bi-directional LSTM

To implement B-LSTM in TensorFlow, we define two LSTM layers, one for forward and one for backward direction as follows:

```
lstm_cell_fw = tf.contrib.rnn.BasicLSTMCell(lstm_units)
lstm_cell_bw = tf.contrib.rnn.BasicLSTMCell(lstm_units)
```

In the previous implementation, we unrolled the LSTM using a dynamic RNN. TensorFlow provides a similar routine for bidirectional LSTM which we can use as follows:

```
(output_fw, output_bw), state =
tf.nn.bidirectional_dynamic_rnn(lstm_cell_fw, lstm_cell_bw, data,
    dtype=tf.float32)
context_rep = tf.concat([output_fw, output_bw], axis=-1)
context_rep_flat = tf.reshape(context_rep, [-1, 2*lstm_units])
```

Now, we initialize weights and bias like before (note, weight has twice the number of `lstm_units` as before, one for each directional layer of LSTM):

```
weight = tf.Variable(tf.truncated_normal([2*lstm_units, num_classes]))
bias = tf.Variable(tf.constant(0.1, shape=[num_classes]))
```

Now, you can generate predictions based on the current value of weights and compute a loss value. In the previous example, we computed a cross entropy loss. For sequence labeling, it is often useful to have a **conditional random field (CRF)**-based loss function. You can define these loss functions as follows:

```
prediction = tf.matmul(context_rep_flat, weight) + bias
scores = tf.reshape(prediction, [-1, max_sequence_length, num_classes])
log_likelihood, transition_params = tf.contrib.crf.crf_log_likelihood(
    scores, labels, sequence_lengths)
loss_crf = tf.reduce_mean(-log_likelihood)
```

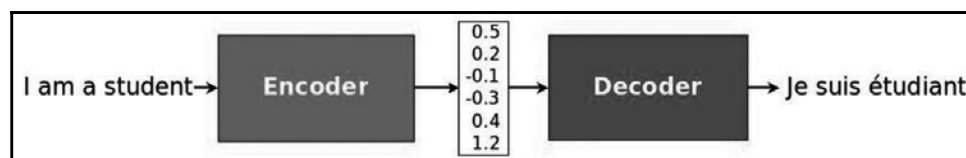
The model can now be trained as follows:

```
optimizer = tf.train.AdamOptimizer().minimize(loss_crf)
```

Machine translation

Machine translation is one of the most recent success stories of NLU. The goal of this problem is to take a text sentence in a source language, such as English and convert it into the same sentence in a given target language, such as Spanish. Traditional methods of solving this problem relied on using phrase-based models. These models typically chunk the sentences into shorter phrases and translate each of these phrases one by one into a target language phrase.

Though translation at phrase-level works reasonably well, when you combine these translated phrases into the target language to generate a fully translated sentence, you find occasional choppiness, or *disfluency*. To avoid this limitation of phrase-based machine translation models, the **neural machine translation (NMT)** technique is proposed, which utilizes a variant of RNN to solve this problem:

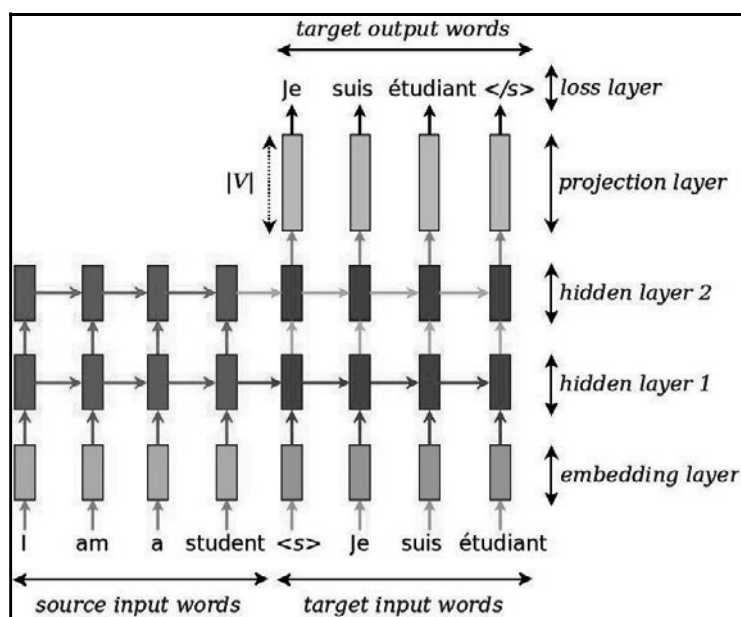


Neural machine translation core idea (Source: <https://github.com/tensorflow/nmt>)

The core idea of an NMT can be described in figure *Neural machine translation core idea*. It consists of two parts: (a) **Encoder** and (b) **Decoder**.

The role of the encoder is to take a sentence in the source language and convert it into a vector representation (also known as **thought vector**) that captures the overall semantics and meaning of the sentence. This vector representation is then inputted to a decoder, which then decodes this into a target language sentence. As you can see, this problem is a natural fit for a many-to-many RNN architecture. In the previous application example for sequence labeling, we introduced the concept of B-LSTM that can map an input sequence to a set of output tokens. Even B-LSTM is unable to map an input sequence to an output sequence. Hence, to solve this problem with an RNN architecture, we introduce another variant of RNN also known as the *Seq2Seq* model.

Figure Neural machine translation architecture with Seq2Seq model describes the core architecture of a Seq2Seq model applied for the task of NMT:



Neural machine translation architecture with Seq2Seq model (Source: <https://github.com/tensorflow/nmt>)

A Seq2Seq model, as shown in Figure Neural machine translation architecture with Seq2Seq model is essentially composed of two groups of RNNs, Encoders and Decoders. Each of these RNNs may be composed of either uni-directional LSTM or bidirectional LSTM, may be composed of multiple hidden layers, and may either use LSTM or GRU as its basic cell unit type. As shown, an encoder RNN (shown on left) takes source words as its inputs and projects it onto two hidden layers. These two hidden layers when moved across the time step, invoke the decoder RNN (shown on right) and project the output onto a projection and loss layer to generate the most likely candidate words in the target language.

To implement Seq2Seq in TensorFlow, we define a simple LSTM cell for both encoding and decoding and unroll the encoder with a dynamic RNN module as follows:

```
lstm_cell_encoder = tf.nn.rnn_cell.BasicLSTMCell(lstm_units)
lstm_cell_decoder = tf.nn.rnn_cell.BasicLSTMCell(lstm_units)
encoder_outputs, encoder_state = tf.nn.dynamic_rnn(lstm_cell_encoder,
encoder_data, sequence_length=max_sequence_length, time_major=True)
```

In case you want to use bi-directional LSTM for this step, you can do that as follows:

```
lstm_cell_fw = tf.contrib.rnn.BasicLSTMCell(lstm_units)
lstm_cell_bw = tf.contrib.rnn.BasicLSTMCell(lstm_units)
bi_outputs,
encoder_state = tf.nn.bidirectional_dynamic_rnn(lstm_cell_fw,
                                                lstm_cell_bw,
                                                encoder_data,
                                                sequence_length=
                                                max_sequence_length,
                                                time_major=True)

encoder_outputs = tf.concat(bi_outputs, -1)
```

Now we need to perform a decoding step to generate the most likely candidate words in the target language (hypotheses). For this step, TensorFlow provides a `dynamic_decoder` function under the `Seq2Seq` module. We use it as follows:

```
training_helper = tf.contrib.seq2seq.TrainingHelper(decoder_data,
                                                    decoder_lengths, time_major=True)
projection_layer = tf.python.layers.core.Dense(target_vocabulary_size)
decoder = tf.contrib.seq2seq.BasicDecoder(decoder_cell,
                                           training_helper, encoder_state, output_layer=projection_layer)
outputs, state = tf.contrib.seq2seq.dynamic_decode(decoder)
logits = outputs.rnn_output
```

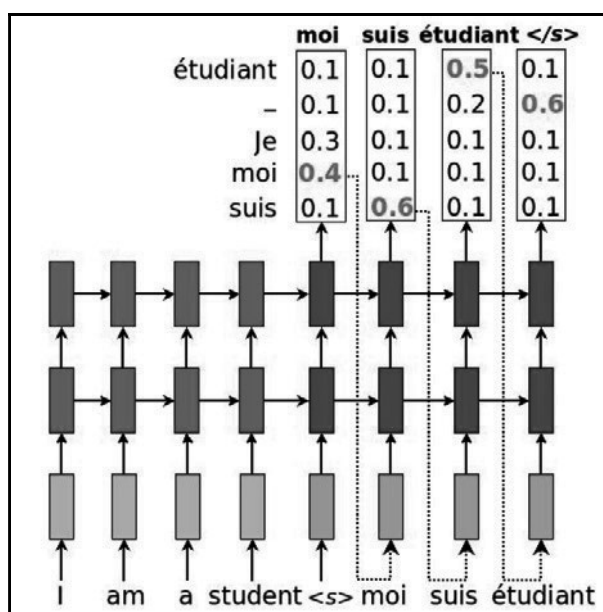
Lastly, we define a loss function and train the model to minimize the loss:

```
loss =
tf.reduce_sum(tf.nn.softmax_cross_entropy_with_logits(logits=prediction,
                                                       labels=labels))
optimizer = tf.train.AdamOptimizer().minimize(loss)
```

Seq2Seq inference

During the inference phase, a trained Seq2Seq model gets a source sentence. It uses this to obtain an `encoder_state` which is used to initialize the decoder. The translation process starts as soon as the decoder receives a special symbol, `<s>`, denoting the start of the decoding process.

The decoder RNN now runs for the current time step and computes the probability distribution of all the words in the target vocabulary as defined by the `projection_layer`. It now employs a greedy strategy, where it chooses the most likely word from this distribution and feeds it as the target input word in the next time step. This process is now repeated for another time step until the decoder RNN chooses a special symbol, `</s>`, which marks the end of translation. Figure *Neural machine translation decoding with greedy search over Seq2Seq model* illustrates this greedy search technique with an example:



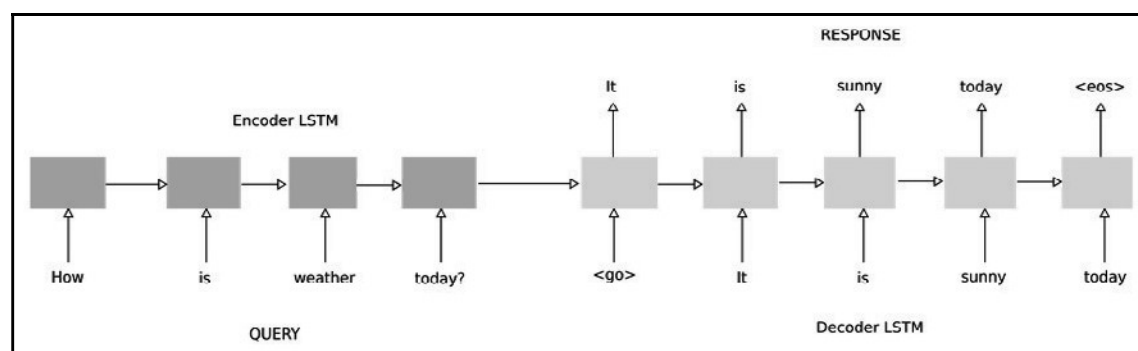
Neural machine translation decoding with greedy search over Seq2Seq model (Source: <https://github.com/tensorflow/nmt>)

To implement this Greedy Search strategy in TensorFlow, we use the `GreedyEmbeddingHelper` function in `Seq2Seq` module as follows:

```
helper = tf.contrib.seq2seq.GreedyEmbeddingHelper(decoder_data,
    tf.fill([batch_size], "<s>"), "</s>")
decoder = tf.contrib.seq2seq.BasicDecoder(
    decoder_cell, helper, encoder_state,
    output_layer=projection_layer)
# Dynamic decoding
num_iterations = tf.round(tf.reduce_max(max_sequence_length) * 2)
outputs, _ = tf.contrib.seq2seq.dynamic_decode(
    decoder, maximum_iterations=num_iterations)
translations = outputs.sample_id
```

Chatbots

Chatbots are another example of an application that lends itself very well to RNN models. Figure *Sequence tagging with traditional LSTM* shows an example of a chatbot application built with the Seq2Seq model, which was described in the preceding section. Chatbots can be understood as a special case of machine translation, where the target language is replaced with a vocabulary of responses for each possible question in the chatbot knowledge base:



Chatbot with Seq2Seq LSTM

Summary

In this chapter, we introduced some core deep learning models for understanding text. We described the core concepts behind sequential modeling of textual data, and what network architectures are more suited to this type of data processing. We introduced basic concepts of recurrent neural networks (RNN) and showed why they are difficult to train in practice. We describe LSTM as a practical form of RNN and sketched their implementation using TensorFlow. Finally, we covered a number of natural language understanding applications that can benefit from the application of various RNN architectures.

In next chapter, chapter 7, we will look at how deep learning techniques can be applied to tasks involving both NLP and images.